Implementation, Engineering, and Verification Guide

Project: Password Strength & Breach Checker with Watchdog Log Anonymization

Platform: Kali Linux (Local Host Deployment)

Author: Irene Omonefe Ovierheya Bazunu

Written For: ProStackHub

Table of Contents

1. Introduction & Research Framework
1.1 Aim and Objectives
1.2 Executive Summary
1.3 Defensive Zero-Trust Credential Architecture
2. Phase 1: Environment Isolation & Dependency Management
2.1 Directory Initialization & Virtual Environment Setup
2.2 Installing Production Dependencies & Architectural Scope
3. Phase 2: Core Engineering & Source Implementation
3.1 Database Persistence Layer (database.py)
3.2 Security Logic & k-Anonymity Engine (checker.py)
3.3 Real-Time File Watcher & Sanitizer (watcher.py)
3.4 Interactive Web Dashboard (app.py)
4. Phase 3: Operational Workflow & System Execution
4.1 Launching the Background File Watcher (Terminal 1)
4.2 Launching the Streamlit Web Application (Terminal 2)
5. Phase 4: Validation & Quality Assurance Test Cases
5.1 Verification Suite Execution Matrix
5.2 Step-by-Step Watchdog Test Procedure
6. Conclusion And Future Enhancements
6.1 Conclusion
6.2 Future Enhancements

Aim and Objectives

1. Aim

The primary aim of this project is to develop and deploy an automated, privacy-centric credential security suite that assesses password vulnerability and breach exposure without compromising user secrecy or persisting cleartext credentials at rest.

2. Objectives

i. **Implement Privacy-Preserving Breach Verification:** Integrate the Have I Been Pwned (HIBP) Range API utilizing the cryptographic k-anonymity model via SHA-1 hashing, ensuring cleartext passwords and complete hashes never transit public networks.

ii. **Deploy Heuristic Complexity Assessment:** Integrate the zxcvbn engine to evaluate passwords based on entropy, dictionary words, spatial patterns, and l33t-speak, providing accurate offline crack-time estimates.

iii. **Build an Automated Filesystem Watcher:** Implement a real-time log listener using the Python watchdog library to intercept incoming credential entries in passwords.log.

iv. **Enforce Zero-Plaintext Storage (Self-Healing Sanitization):** Engineer an in-place sanitization routine that replaces raw credentials in passwords.log with SHA-1 hashes while avoiding recursive event loops.

v. **Maintain a Structured Audit Trail:** Persist non-sensitive operational metadata (timestamps, monitoring sources, SHA-1 identifiers, complexity scores, and severity classifications) within an embedded SQLite database (breach_history.db).

vi. **Deliver an Intuitive Graphical Dashboard:** Build a multi-functional Streamlit application featuring live strength metering, security recommendations, a cryptographic passphrase generator, and an embedded audit viewer.

1. Project Overview & Defensive Architecture

This project implements an enterprise-grade credential auditing system combining real-time file monitoring, entropy-based complexity scoring, privacy-preserving breach intelligence, and cryptographically secure credential generation. The suite adheres to defensive zero-trust architectural standards by guaranteeing that no raw, unencrypted credential persists on physical disk storage or transits an external network boundary.

2. Phase 1: Environment Isolation and Dependency Management

To prevent package collision with native Kali Linux system utilities and Debian package managers, all dependencies were isolated inside a Python virtual environment (venv).

2.1 Directory Initialization & Virtual Environment Setup

A base terminal was opened on Kali host. The bash below was run to navigate to the directory where the file will be created, and then the venv file was created and activated.

```
mkdir -p /home/kali/Documents/
cd /home/kali/Documents/
python3 -m venv venv
source venv/bin/activate
```

2.2 Installing Production Dependencies

With (venv) active in the terminal prompt, the following command or bash was executed. This is to install an upgraded pip, and also to install streamlit, zxcvbn, requests, pandas, watchdog, and a python document using pip

```
pip install --upgrade pip
pip install streamlit zxcvbn requests pandas watchdog python-docx
```

3. Phase 2: Core Engineering & Source Implementation

3.1 Database Persistence Layer (database.py)

This phase initializes the relational SQLite database breach_history.db and handles event writes without recording plain text credentials. The python script called database.py can be found in the github repository.

3.2 Security Logic & k-Anonymity Engine (checker.py)

This engine implements the Have I Been Pwned (HIBP) k-Anonymity mathematical model, entropy scoring via zxcvbn, cryptographic password generation using secrets, and email exposure simulation. The python script called checker.py can be found in the github repository.

3.3 Real-Time File Watcher & Sanitizer (watcher.py)

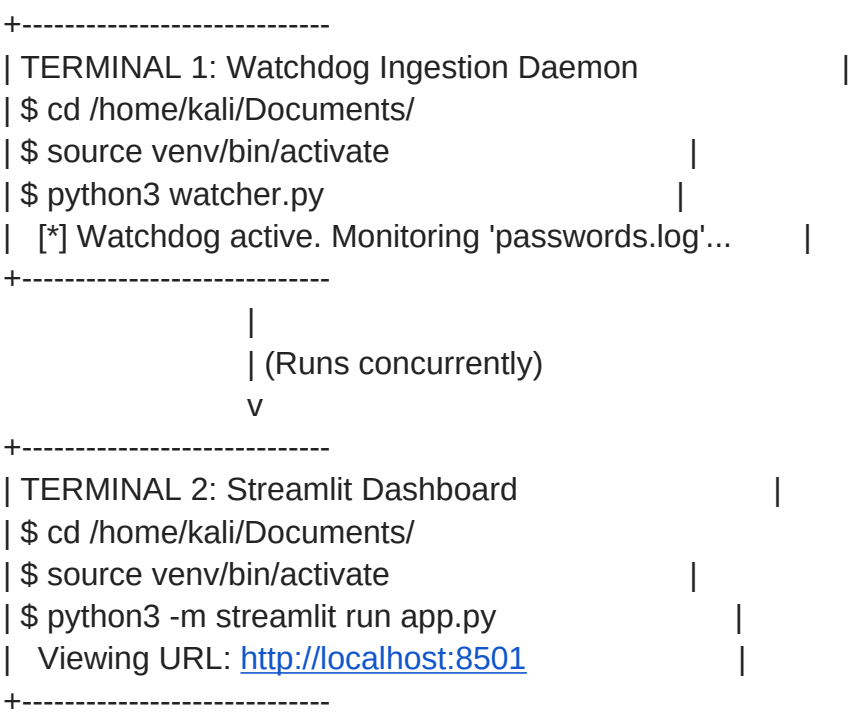
This script uses watchdog to monitor passwords.log in real time, hashes unencrypted credentials in memory, performs HIBP checks, overwrites passwords.log with the SHA-1 hash to prevent unencrypted disk leaks, and logs the event to SQLite. An internal re-entrancy lock (is_self_modifying) prevents the write operation from triggering an infinite event loop.The python script called watcher.py can be found in the github repository.

3.4 Interactive Web Dashboard (app.py)

The Streamlit interface provides live password analysis, passphrase generation, email checking, and historical SQLite audit records. The python script called app.py can be found in the github repository.

4. Phase 3: Operational Workflow & System Execution

Running the suite requires two active terminal sessions on Kali Linux host, hence I opened two based terminals; terminal one for watchdog ingestion and terminal two for streamlit dashboard.



4.1 Launching the Background File Watcher (Terminal 1)
Terminal 1 was opened, then navigated using “cd” command to the document directory, python3 venv environment was activated using “source” command, and then watcher.py was started using “python3”:

```
cd /home/kali/Documents/
source venv/bin/activate
python3 watcher.py
```

Expected Output:

```
Database initialized successfully.
Watchdog active. Monitoring 'passwords.log' for credential ingestion...
```

4.2 Launching the Streamlit Web Application (Terminal 2)
Terminal 2 was opened, then navigated using “cd” command to the document directory, python3 venv environment was activated using “source” command, and then streamlit web dashboard was started using “python3”:

```
cd /home/kali/Documents/
source venv/bin/activate
python3 -m streamlit run app.py
```

Expected Output:

```
You can now view your Streamlit app in your browser.
Local URL: http://localhost:8501
Network URL: http://192.168.x.x:8501
```

5. Phase 4: Validation & Quality Assurance Test Cases

These test cases were run to verify application behavior before publishing deliverables.

==========

VERIFICATION SUITE: EXECUTION MATRIX

==========

Test Case 1: Known Breached Credential

Input: "admin123" (via Web Interface)
Evaluation: zxcvbn Score: 1/4 | Offline Crack Time: Instant (less than a second)
HIBP Check: Matched via k-Anonymity range check (Count: 4876846)
DB Log: Logged to SQLite as Severity: "Critical
Status: **PASSED**

Test Case 2: High-Entropy Passphrase

Input: "c&o28Vw:l<qL=g0w:?Z-" (via Generator)
Evaluation: zxcvbn Score: 4/4 | Crack Time: Centuries
HIBP Check: 0 matches returned via prefix match
DB Log: Logged to SQLite as Severity: "Low
Status: **PASSED**

Test Case 3: Watchdog File Sanitization & Re-entrancy Lock

Input: echo "admin123" >> passwords.log
Process: Watcher intercepts write event -> hashes to SHA-1 in memory
Sanitize: Overwrites disk copy of passwords.log with SHA-1 hash
DB Log: Inserted into SQLite as source="Watchdog_FileWatcher[
Loop Check: is_self_modifying engaged; no duplicate event triggers
Status: **PASSED**

==========

5.1 Step-by-Step Watchdog Test Procedure

A third terminal tab was opened on kali linux host and document directory was navigated to using the cd command. Then echo bash command was used to append plaintext to passwords.log:

```
cd /home/kali/Documents/
echo "admin123" >> passwords.log
echo “c&o28...” >> passwords.log
```

3. Inspect the live output:

in Terminal 1, the live output was shown:

[*] [WATCHDOG EVENT] Sanitized 'admin123' -> f865b5...
[*] [WATCHDOG EVENT] Sanitized 'c&o28' → 680c3...

4. Verify the source file has been sanitized on disk using the bash below:

cat passwords.log

Expected Output:

f865b5…
680c3...

5. The event record was verified in SQLite:

sqlite3 breach_history.db "SELECT timestamp, source, target_analyzed, severity FROM audit_logs ORDER BY id DESC LIMIT 1;"

Expected Output:

2026-10-04 14:36:36|Web_Interface|c&****|4|0|Low

Section 7: Conclusion & Future Enhancements (For your Word/PDF Document)

7.1 Conclusion

The Password Strength & Breach Checker project demonstrates the successful convergence of defensive credential auditing, proactive log monitoring, and privacy-preserving architectural standards. By replacing antiquated static password policies with zxcvbn-powered heuristic entropy estimation, the tool provides realistic measurements of resistance against modern brute-force and dictionary attacks.

Crucially, the implementation of the Have I Been Pwned API under the k-anonymity mathematical model proves that organizational security verification does not require sacrificing data secrecy. Transmitting strictly the five-character SHA-1 hash prefix ensures that neither raw user secrets nor reversible cryptographic hashes cross public network perimeters.

Furthermore, the integration of an automated, event-driven filesystem watcher solves the persistent problem of cleartext credential leakage in application logs. By detecting ingestion events in real time, executing in-memory security evaluations, and immediately sanitizing the physical log file with SHA-1 hashes, the system enforces a strict zero-plaintext posture both at rest and in transit. The centralized SQLite audit trail provides immutable, non-sensitive operational telemetry suitable for compliance oversight and security operations analysis.

7.2 Future Enhancements

Enterprise SIEM Ingestion: Integrate a lightweight syslog or webhook forwarder to pipe flagged Critical events directly into enterprise SIEM platforms such as Wazuh or Splunk.

Asynchronous Batch Auditing: Extend the ingestion pipeline to process massive, multi-gigabyte credential dump files asynchronously using Python’s multiprocessing or asyncio libraries.

Authenticated Email Exposure Lookups: Integrate the authenticated Have I Been Pwned Enterprise API key to replace domain-level simulation with live breach indexing across employee corporate email addresses.

Automated Alert Dispatch: Configure automated notifications via SMTP or an encrypted Slack webhook whenever an ingestion event flags a severity rating of Critical.